

TASK6 智能决策者：用机器学习定制专属策略

主任务选用季度股票横截面数据，因为这类数据可以直接检验“用当期因子预测下期收益排名”的选股逻辑。模型输出随后转换为每季持有预测前 30 只股票的策略，并与市场等权收益比较。附加题使用平安银行日线数据，考察分类概率能否经过双阈值和动态仓位规则形成可执行的择时策略。

一、基于机器学习模型的交易策略

1. 核心理念

机器学习交易策略从历史数据中学习输入变量与未来结果之间的关系。输入变量记为 X ，在本作业中是季末可观察的估值、规模和成长因子；预测目标记为 Y ，主任务对应下一季度的股票收益排名。模型给出的分数、排名或概率只表示相对信号强弱，必须配合持股数、仓位、阈值、调仓频率和风险控制，才能转换为实际决策。

主策略在每个季末计算股票因子，预测下季度收益在当期股票池中的相对位置，再等权买入前 30 只。到下一个季末时，新数据会产生新排名和新持仓。这一设计把季末可得信息、未来收益排名、Top 30 选股和下季度实现收益连接起来，使模型目标与交易目标保持一致。

2. 优点和缺点

方面	优点	缺点
信息处理	可以同时利用多个因子，树模型还能捕捉非线性和交互关系	当数据本身没有稳定信息时，增加模型复杂度并不会自动产生有效信号
规则一致性	同样的数据和参数可以复现同样的决策	数据泄漏、幸存者偏差和反复试验可能制造虚假的稳定性
模型更新	可以通过滚动训练吸收新数据	市场状态会变化，早期样本中有效的关系未必能够延续
交易执行	概率可以转换为动态仓位，也能加入双阈值、止损和止盈	手续费、滑点、停牌、涨跌停和容量限制会使实盘结果低于理想回测
解释性	线性系数和特征重要性可以帮助检查模型	重要性只反映模型对变量的依赖，不能直接解释为因果关系

二、量化交易模型中的自变量和应变量

1. 常见自变量因子

自变量是模型用来产生预测的已知信息。量化交易中的自变量通常以“因子”表示，每个因子都要在决策时点已经可得，否则会引入未来信息。本作业的主样本以财务和市场估值因子为主，附加题则使用日度动量、趋势、波动和成交量信息。

因子类型	常用指标	基本定义
估值	PE、PB、PS、EV/EBITDA、PCF、股息率	描述股价相对于盈利、净资产、收入或现金流的高低
规模	总市值、流通市值及其对数	表示公司的权益市场价值
质量	ROE、毛利率、资产负债率、经营现金流	衡量盈利能力、财务结构和利润的现金支持
成长	收入、净利润、EPS、资产和现金流增长率	衡量公司经营指标相对上年同期的变化
动量与趋势	过去 1、5、20、60 日收益、均线偏离、MACD、RSI	描述价格在近期的方向、速度和超买超卖程度

因子类型	常用指标	基本定义
风险与流动性	波动率、ATR、Beta、换手率、成交量比、买卖价差	描述价格不确定性、交易活跃度和交易难度

2. 常见应变量

应变量是模型希望预测的未来结果，其时间窗口必须与投资持有期对齐。回归任务可以预测未来 h 期原始收益 $r_{i,t \rightarrow t+h}$ 、相对基准的超额收益，或股票在当期横截面中的收益排名；分类任务则可以预测未来收益是否为正、是否跑赢基准，或是否进入股票池前 30%。风险模型常把未来波动率、最大回撤或极端下跌事件作为目标。

主任务要解决股票排序问题，因此回归应变量定义为同季度 `Next_Ret` 的百分位排名，再居中到 $[-0.5, 0.5]$ 。这种转换保留了股票的高低顺序，同时限制了单个极端收益对拟合的影响。逻辑回归虽然名称中含有“回归”，实际上属于分类模型，其 Y 定义为 `Next_Ret` 是否高于同季度中位数。所有模型最终都按预测分数排列股票，组合回测仍使用未转换的 `Next_Ret` 计算实现收益。

三、Python 编程实现

1. 加载已存储的模型样本

主任务读取课程提供的 `model_data.csv`，每行表示某只股票在一个季末的因子和下期收益。整理后的样本包含 39,616 条股票—季度记录，涉及 4,281 只股票和 10 个季度。加载后先检查股票代码与日期组合是否唯一、数值列是否缺失，再进行因子衍生和时间划分。固定随机种子为 42，使随机森林等模型在重复运行时产生一致结果，便于核对模型比较和回测指标。

原始文件没有提供财务报表的实际披露日期，也缺少历史成分股、ST、停牌和流动性标记。因此，季末财务因子能否在当时完整取得无法独立验证，样本中也可能存在时点可得性偏差、幸存者偏差和实际交易限制。后文的模型指标与组合收益主要用于课程中的方法比较，不能直接外推为未来实盘收益，也不构成投资建议。

```
[1]: from pathlib import Path
import json
import sys
import warnings

warnings.filterwarnings("ignore")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from IPython.display import display

START = Path.cwd().resolve()
PROJECT_ROOT = START if (START / "Task6").exists() else START.parent
SCRIPT_DIR = PROJECT_ROOT / "Task6" / "scripts"
if str(SCRIPT_DIR) not in sys.path:
    sys.path.insert(0, str(SCRIPT_DIR))

from main_pipeline import run_main_pipeline
from additional_pipeline import run_additional_pipeline
from plot_results import main as build_plots

pd.set_option("display.max_columns", 20)
pd.set_option("display.float_format", lambda value: f"{value:.4f}")
RUN_PIPELINE = True
TRANSACTION_COST = 0.002
TOP_N = 30
BUFFER_RANK = 50
RANDOM_SEED = 42

if RUN_PIPELINE:
    run_main_pipeline()
```

```
run_additional_pipeline()
build_plots()
print("TASK6 主任务、附加题和 13 张图已重新生成。")
```

[plots] wrote 15 figures to /Users/rebecca/Desktop/量化交易/data/charts/task6
TASK6 主任务、附加题和 13 张图已重新生成。

```
[2]: MAIN_DIR = PROJECT_ROOT / "data" / "task6" / "main"
ADDON_DIR = PROJECT_ROOT / "data" / "task6" / "additional"
CHART_DIR = PROJECT_ROOT / "data" / "charts" / "task6"

main_dataset = pd.read_csv(
    MAIN_DIR / "processed" / "main_model_dataset.csv",
    parse_dates=["Date"], dtype={"Code": "string"}
)
main_quality = json.loads(
    (MAIN_DIR / "metadata" / "data_quality_report.json").read_text(encoding="utf-8")
)

quality_table = pd.DataFrame({
    "检查项目": ["样本行数", "季度数", "股票代码数", "重复股票季度键", "原始缺失单元格", "训练行数", "测试行数"],
    "结果": [
        main_quality["rows"], main_quality["quarter_count"], main_quality["stock_count"],
        main_quality["duplicate_code_date_rows"], main_quality["missing_raw_cells"],
        main_quality["train_rows"], main_quality["test_rows"],
    ],
})
print("表 1: 主任务数据检查")
display(quality_table)

assert main_quality["duplicate_code_date_rows"] == 0
assert main_quality["missing_raw_cells"] == 0
assert main_dataset.duplicated(["Code", "Date"]).sum() == 0
```

表 1: 主任务数据检查

	检查项目	结果
0	样本行数	39616
1	季度数	10
2	股票代码数	4281
3	重复股票季度键	0
4	原始缺失单元格	0
5	训练行数	26953
6	测试行数	12663

图1: 季度样本覆盖与Next_Ret分布

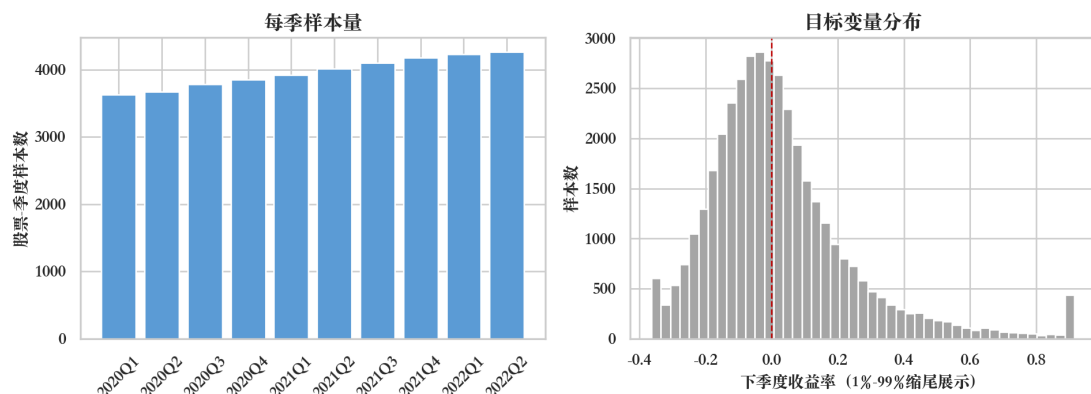


图 1 显示，每季股票记录由 2020Q1 的 3,627 条增加到 2022Q2 的 4,262 条，各季样本规模总体稳定，能够支持横截面排名。Next_Ret 分布存在明显右长尾，最大值超过 600%，如果直接拟合原始收益，少数极端值容易放大均方误差并改变模型系数。因此训练目标改用季度收益排名，回测阶段仍保留原始实现收益，以确保组合结果反映真实收益差异。

2. 衍生自变量并设计应变量

原始数据含 19 项估值、规模、股息和成长指标。PE、PCF 等比率可以为负，不同指标的数值范围也相差较大，因此先在每个季度内转换为横截面百分位排名。这项处理把原始数值转换为股票在同期样本中的相对位置，便于模型比较不同因子的排序信息：

$$RankX_{i,t} = PctRank_t(x_{i,t}) - 0.5$$

处理后的特征主要位于 $[-0.5, 0.5]$ 。19 个排名特征之外，又分别对价值、成长、利润增长和现金流信息取均值，得到 4 个复合因子，最终 X 共 23 项。这些复合因子用于减少同类指标的重复波动，同时保留个股在每个财务维度上的相对位置。回归模型的 Y 同样转换为当季度内 $Next_Ret$ 的百分位排名：

$$Y_{i,t}^{rank} = PctRank_t(Next_Ret_{i,t}) - 0.5$$

逻辑回归的二元 Y 则表示个股收益是否高于同季度中位数，取值 1 表示高于中位数，取值 0 表示未超过。中位数标签使每个季度的两类样本数量大致均衡，因此逻辑回归的输出概率可以解释为个股跑赢同期中位数的可能性：

$$Y_{i,t}^{class} = 1[Next_Ret_{i,t} > Median_t(Next_Ret)]$$

季度横截面排名特征与原始收益点预测的口径并不完全一致，因为季度整体涨跌会混入误差，却不会改变当期股票的相对顺序。将 X 和 Y 都对齐到横截面排名后，模型评价可以直接服务于 Top 30 选股。

[3]:

```
split_table = (
    main_dataset.groupby(["Split", "Date"], as_index=False)
    .agg(样本数=("Code", "size"), 平均原始收益=("Next_Ret", "mean"), 排名目标均值=("Next_Ret_Rank", "mean"))
)
split_table["季度"] = split_table["Date"].dt.year.astype(str) + "Q" + split_table["Date"].dt.quarter.astype(str)
split_table["数据集"] = split_table["Split"].map({"train": "训练集", "test": "测试集"})
print("表 2: 7:3 时间划分与应变量")
display(split_table[["数据集", "季度", "样本数", "平均原始收益", "排名目标均值"]])
print("模型特征数: ", len(main_quality["model_features"]))
print("排名目标范围: ", main_dataset["Next_Ret_Rank"].min(), "至", main_dataset["Next_Ret_Rank"].max())
```

表 2: 7:3 时间划分与应变量

	数据集	季度	样本数	平均原始收益	排名目标均值
0	测试集	2021Q4	4173	-0.0837	0.0001
1	测试集	2022Q1	4228	0.0107	0.0001
2	测试集	2022Q2	4262	-0.0920	0.0001
3	训练集	2020Q1	3627	0.1207	0.0001
4	训练集	2020Q2	3667	0.0948	0.0001
5	训练集	2020Q3	3781	0.0047	0.0001
6	训练集	2020Q4	3853	-0.0178	0.0001
7	训练集	2021Q1	3916	0.0990	0.0001
8	训练集	2021Q2	4011	0.0447	0.0001
9	训练集	2021Q3	4098	0.1136	0.0001

模型特征数: 23

排名目标范围: -0.4997653683716565 至 0.5

图2：严格按时间排列的7:3划分



图 2 给出了训练集和测试集的时间边界。2020Q1 至 2021Q3 的 7 个季度用于训练，共 26,953 条记录；2021Q4 至 2022Q2 的 3 个季度仅用于最终测试，共 12,663 条记录。时序数据如果随机打乱，训练集可能含有测试时点之后的市场状态，因此本作业保留原始时间顺序。参数比较进一步限定在训练期内，通过扩展窗口分别验证 2021Q1、2021Q2 和 2021Q3，使测试集在模型与参数确定前保持未见。

3. 划分训练集、测试集，构建并训练模型

模型比较包括普通线性回归、Ridge 回归、逻辑回归、决策树、随机森林和直方图梯度提升。线性回归提供结构最简单的排名基准；Ridge 在损失函数中增加系数惩罚，用于减少高相关因子导致的系数波动；决策树、随机森林和梯度提升用于检验非线性和变量交互是否能提供额外信息。逻辑回归输出个股收益高于季度中位数的概率，再用该概率完成统一排序。

候选参数只在训练期扩展窗口内比较。Rank IC 是预测排名与实际收益排名的 Spearman 相关系数，取值大于 0 表示排序方向一致，数值越高表示排序匹配程度越强。各验证季度 Rank IC 的均值用于选参数；若简单模型与最高验证 IC 的差距不超过 0.01，则优先保留结构更简单的模型，以降低样本变化时的不稳定性。

回归模型同时使用 R^2 检查点预测误差。 R^2 表示模型对应变量波动的解释程度，大于 0 表示预测误差低于直接使用样本均值。分类模型使用 AUC 评价正类与负类的排序能力，AUC 为 0.5 相当于随机排序，越接近 1 表示正类更稳定地排在负类之前。这两个指标只用于检查模型，交易决策仍根据横截面排名和组合收益确定。

```
[4]: model_metrics = pd.read_csv(MAIN_DIR / "processed" / "main_model_metrics.csv")

candidate_short = {
    "ordinary least squares": "OLS",
    "alpha=10": "alpha=10",
    "C=0.01": "C=0.01",
    "depth=4,leaf=250": "d4,l250",
    "depth=5,leaf=20,features=0.5": "d5,l20,f0.5",
    "leaves=15,l2=10": "l15,L2=10",
}

def native_result(row):
    if row["task_type"] == "classification":
        return f"测试 AUC={row['auc']:.3f}"
    return f"测试 R²={row['r2']:.3f}"

model_table = pd.DataFrame({
    "模型": model_metrics["model_label"],
    "任务": model_metrics["task_type"].map({"rank_regression": "排名回归", "classification": "分类"}),
    "入选参数": model_metrics["selected_candidate"].map(candidate_short),
    "验证平均 IC": model_metrics["validation_mean_ic"],
    "本任务指标": model_metrics.apply(native_result, axis=1),
    "测试平均 IC": model_metrics["mean_test_ic"],
    "主策略": model_metrics["strategy_model"].map({True: "是", False: ""}),
})

print("表 3：六种模型的样本外结果")
display(model_table)
```

表 3：六种模型的样本外结果

模型	任务	入选参数	验证平均 IC	本任务指标	测试平均 IC	主策略
----	----	------	---------	-------	---------	-----

0	线性回归	排名回归	OLS	0.1288	测试 $R^2=0.068$	0.2733	是
1	Ridge 回归	排名回归	alpha=10	0.1315	测试 $R^2=0.068$	0.2741	
2	逻辑回归	分类	C=0.01	0.1258	测试 AUC=0.633	0.2625	
3	决策树	排名回归	d4,1250	0.1137	测试 $R^2=0.050$	0.2354	
4	随机森林	排名回归	d5,120,f0.5	0.1298	测试 $R^2=0.062$	0.2595	
5	梯度提升	排名回归	115,L2=10	0.1272	测试 $R^2=0.065$	0.2576	

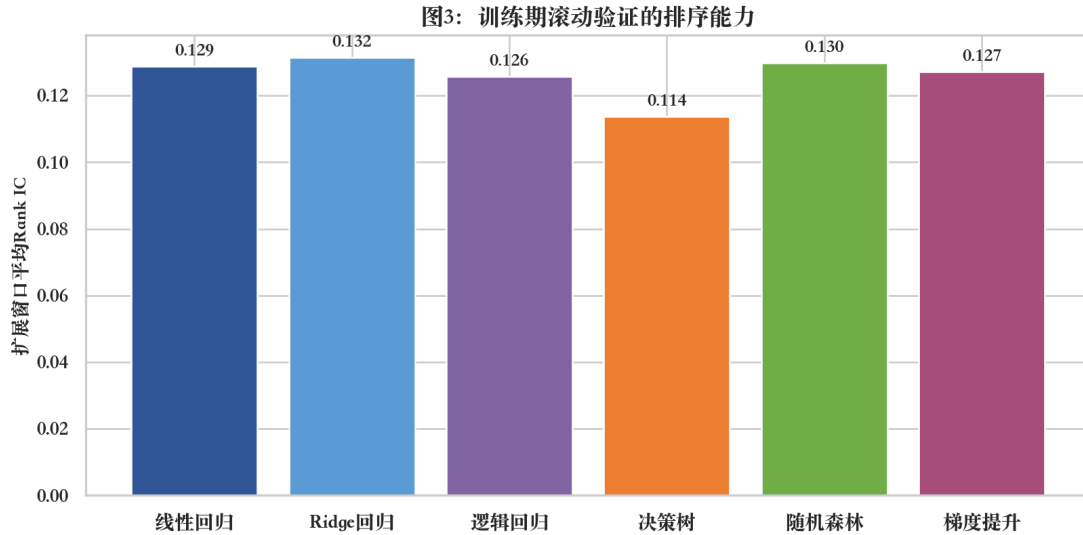


图 3 中六种模型的验证平均 Rank IC 介于 0.114 和 0.132 之间, 说明训练期内的排序信号为正, 但模型之间差距较小。Ridge 的验证 IC 最高, 为 0.132, 普通线性回归为 0.129, 差值 0.003 低于预先设定的 0.01 简约容差。这一结果没有提供足够证据支持额外复杂度, 因此主策略按训练期规则选择普通线性回归。

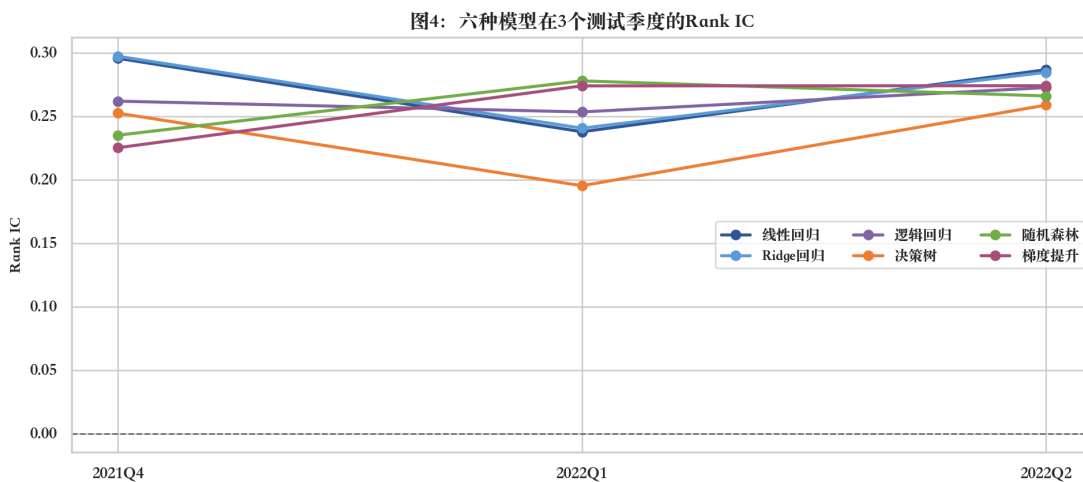


图 4 中 18 个“模型-季度”Rank IC 均为正, 与训练期验证方向一致。线性回归的测试平均 IC 为 0.273, Ridge 为 0.274, 决策树最低, 为 0.235; 五个排名回归模型的测试 R^2 介于 0.050 和 0.068 之间, 逻辑回归的 AUC 为 0.633。排序指标高于 0、分类 AUC 高于 0.5, 表明模型在三个测试季度中保留了正向信息; 由于测试期只有三季, 指标仍需结合组合收益和换手结果判断。

4. 基于模型建立交易策略

模型排序要转换为统一的组合规则, 才能比较不同模型的实际经济效益。每个测试季度先按模型分数从高到低排序, 再等权持有前 30 只, 组合毛收益是 30 只入选股票 Next_Ret 的算术平均。市场基准使用当季度全部样本股票等权平均, 使两者具有相同的时间和收益口径:

$$R_{t+1}^{Top30} = \frac{1}{30} \sum_{i \in Top30_t} Next_Ret_{i,t}$$

$$R_{t+1}^{Market} = \frac{1}{N_t} \sum_{i=1}^{N_t} Next_Ret_{i,t}$$

换手率衡量调仓时发生变化的仓位比例。例如，30 只股票中替换 9 只，单边换手率为 30%。交易成本采用单边 20bp，1bp 等于 0.01 个百分点，因 20bp 表示每变动 1 元仓位扣 0.002 元成本。设换手率为 $Turnover_t$ ，组合净收益定义为：

$$R_t^{net} = R_t^{gross} - 0.002 \times Turnover_t$$

首次建仓的换手率记为 100%，之后严格 Top 30 的单边换手率为 $1 - |H_t \cap H_{t-1}|/30$ 。该口径使成本随实际持仓替换而变化，同时保持六种模型的评价条件一致。每季度都重新排序会产生较高换手，所以净收益比毛收益更接近可实现的策略结果，也是后续模型对比和敏感性检验的主要口径。

[5]:

```
quarterly_returns = pd.read_csv(
    MAIN_DIR / "processed" / "main_quarterly_returns.csv", parse_dates=["Date"]
)
main_metadata = json.loads(
    (MAIN_DIR / "metadata" / "model_run.json").read_text(encoding="utf-8")
)
strategy_model = main_metadata["strategy_model"]

main_quarterly = quarterly_returns[
    (quarterly_returns["model"] == strategy_model)
    & (quarterly_returns["portfolio"] == "strict_top30")
].copy()
main_quarterly["季度"] = main_quarterly["Date"].dt.year.astype(str) + "Q" + main_quarterly["Date"].dt.quarter.astype(str)
main_quarterly = main_quarterly[["季度", "gross_return", "net_return", "market_return", "gross_excess", "turnover"]]
main_quarterly.columns = ["季度", "Top30 毛收益", "Top30 净收益", "市场平均", "毛超额", "单边换手率"]
print("表 4: 线性回归 Top 30 在三个测试季度的收益")
display(main_quarterly)
```

表 4: 线性回归 Top 30 在三个测试季度的收益

	季度	Top30 毛收益	Top30 净收益	市场平均	毛超额	单边换手率
0	2021Q4	0.1517	0.1497	-0.0837	0.2355	1.0000
1	2022Q1	0.1625	0.1606	0.0107	0.1517	0.9333
2	2022Q2	0.0612	0.0601	-0.0920	0.1532	0.5667

[6]:

```
holdings = pd.read_csv(
    MAIN_DIR / "processed" / "main_portfolio_holdings.csv",
    parse_dates=["Date"], dtype={"Code": "string"}
)
selected_holdings = holdings[
    (holdings["model"] == strategy_model)
    & (holdings["portfolio"] == "strict_top30")
].sort_values(["Date", "predicted_rank"])

quarter_dates = [pd.Timestamp(date) for date in sorted(selected_holdings["Date"].unique())]
quarter_labels = [f"{date.year}Q{date.quarter}" for date in quarter_dates]
quarter_codes = []
for date in quarter_dates:
    codes = selected_holdings.loc[selected_holdings["Date"] == date, "Code"].tolist()
    quarter_codes.append(["." + ".".join(codes[i:i + 3]) for i in range(0, 30, 3)])

holdings_table = pd.DataFrame({"组别": range(1, 11)})
for label, codes in zip(quarter_labels, quarter_codes):
    holdings_table[label] = codes
print("表 5: 线性回归主策略每个测试季度选中的 30 只股票代码")
display(holdings_table)
```

表 5: 线性回归主策略每个测试季度选中的 30 只股票代码

	组别	2021Q4	2022Q1	2022Q2
0	1	000573、000637、002774	002783、300533、600780	300105、000806、002696

1	2	002160、002615、603095	600387、000525、603086	600287、600387、002358
2	3	002188、002722、002574	002343、000404、002478	603586、002478、002193
3	4	300609、002910、601518	002328、600287、002718	002699、603003、300533
4	5	600569、000726、000965	603585、002133、000726	600780、603086、600565
5	6	000521、002345、300240	600508、002365、603586	000726、000543、605088
6	7	000761、603196、603889	603928、600449、002696	002661、003042、002871
7	8	000404、600113、000822	002635、603329、603803	000616、002856、300495
8	9	600748、603029、300154	002485、600525、002358	000159、000892、603585
9	10	603009、600123、000888	603556、600232、300836	002869、002718、603329

表 5 按预测排名列出三个测试季度的完整选股结果，每列均为 30 只，代码从上到下对应模型分数从高到低。原始样本只提供六位股票代码，没有证券简称，因此表中保留原始代码，不根据外部信息补写名称。最近测试季度为 2022Q2，该列中的 30 只股票构成主策略在当期的实际持仓，也是最终结论中的明确选股结果。

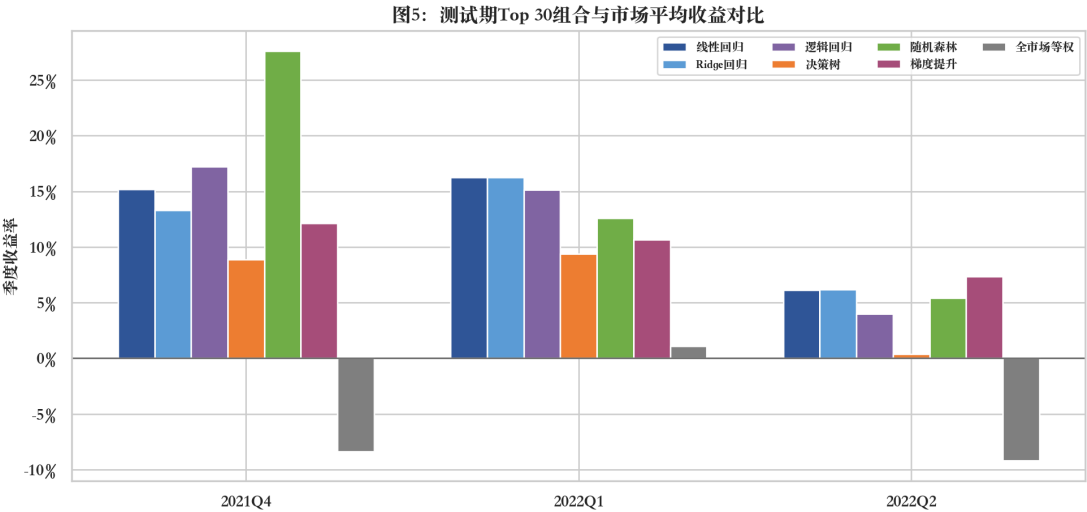


图 5 显示，线性回归 Top 30 在 2021Q4、2022Q1 和 2022Q2 的毛收益分别为 15.17%、16.25% 和 6.12%，同期市场等权收益分别为-8.37%、1.07% 和-9.20%，三个季度的收益差均为正。其他五种模型也使用相同的 Top 30、持有期和市场基准，因而组合差异可以归因于模型排序不同。线性回归的三季表现与正的测试 Rank IC 方向一致，但样本期数较少，还要通过累计净值和成本敏感性检查结果。

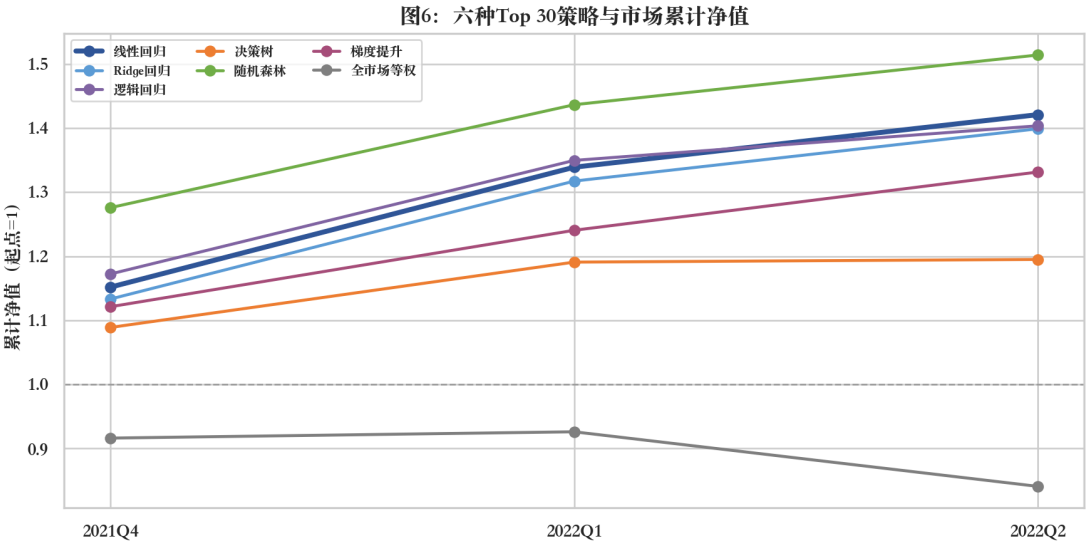


图 6 将三个季度收益按复利连乘。线性回归 Top 30 的累计毛收益为 42.08%，市场等权同期为-15.91%；随机森林在该测试窗口的毛收益最高，为 51.39%。随机森林的收益是测试完成后才观察到

的结果，如果据此更换主模型，测试集就会参与模型选择。主策略因此仍遵循训练期内部验证规则，保留线性回归，并在后续单独评估交易成本的影响。

5. 回测策略，计算指标并绘图

回测指标用于从收益、波动和下跌风险三个方面评估策略。累计收益是各期收益连乘后的总增长；年化收益把当前回测期的复利收益换算为一年口径；年化波动率衡量单期收益的波动程度。夏普比率用平均收益除以收益标准差并换算为年化口径，本作业假设无风险利率为 0，数值越高表示每承担一单位波动获得的平均收益越高。最大回撤是净值从历史高点下降到之后低点的最大幅度，胜率则是收益为正的期数占全部期数的比例。

表 6 按相同口径汇总了六种 Top 30 策略的收益、风险和平均换手率。主测试集只包含 3 个季度，这会使年化收益、年化波动率和夏普比率对单季变化非常敏感。因此这些年化指标只用于在同一窗口内统一比较模型，不解释为策略的长期稳定参数。

```
[7]: strategy_metrics = pd.read_csv(MAIN_DIR / "processed" / "main_strategy_metrics.csv")

gross_summary = strategy_metrics[
    (strategy_metrics["portfolio"] == "strict_top30")
    & (strategy_metrics["return_type"] == "gross_return")
][["model_label", "average_turnover", "total_return", "annualized_return", "annualized_volatility", "sharpe", "max_drawdown",
    ↪ "win_rate"]].copy()
gross_summary.columns = ["模型", "平均换手", "累计收益", "年化收益", "年化波动", "夏普比率", "最大回撤", "季度胜率"]
print("表 6: 六种 Top 30 策略的毛收益指标")
display(gross_summary)

selected_summary = strategy_metrics[strategy_metrics["model"] == strategy_model][
    ["portfolio", "return_type", "average_turnover", "total_return", "sharpe", "max_drawdown"]
].copy()
selected_summary["组合"] = selected_summary["portfolio"].map({"strict_top30": "严格 Top 30", "buffer_top30_top50": "Top 50 缓冲"})
selected_summary["口径"] = selected_summary["return_type"].map({"gross_return": "毛收益", "net_return": "扣 20bp 后"})
print("表 7: 主策略的交易成本和缓冲换仓检查")
display(selected_summary[["组合", "口径", "average_turnover", "total_return", "sharpe", "max_drawdown"]])
```

表 6：六种 Top 30 策略的毛收益指标

	模型	平均换手	累计收益	年化收益	年化波动	夏普比率	最大回撤	季度胜率
0	决策树	0.7333	0.1950	0.2681	0.1014	2.4453	0.0000	1.0000
2	梯度提升	0.9222	0.3314	0.4646	0.0491	8.1778	0.0000	1.0000
6	线性回归	0.8333	0.4208	0.5973	0.1112	4.5013	0.0000	1.0000
8	逻辑回归	0.8333	0.4035	0.5713	0.1421	3.4096	0.0000	1.0000
10	随机森林	0.9111	0.5139	0.7384	0.2262	2.6856	0.0000	1.0000
12	Ridge 回归	0.8222	0.3989	0.5645	0.1033	4.6153	0.0000	1.0000

表 7：主策略的交易成本和缓冲换仓检查

	组合	口径	average_turnover	total_return	sharpe	max_drawdown
4	Top 50 缓冲	毛收益	0.7778	0.3735	2.9172	0.0000
5	Top 50 缓冲	扣 20bp 后	0.7778	0.3679	2.8990	0.0000
6	严格 Top 30	毛收益	0.8333	0.4208	4.5013	0.0000
7	严格 Top 30	扣 20bp 后	0.8333	0.4146	4.4779	0.0000

图7：20bp成本与缓冲换仓敏感性

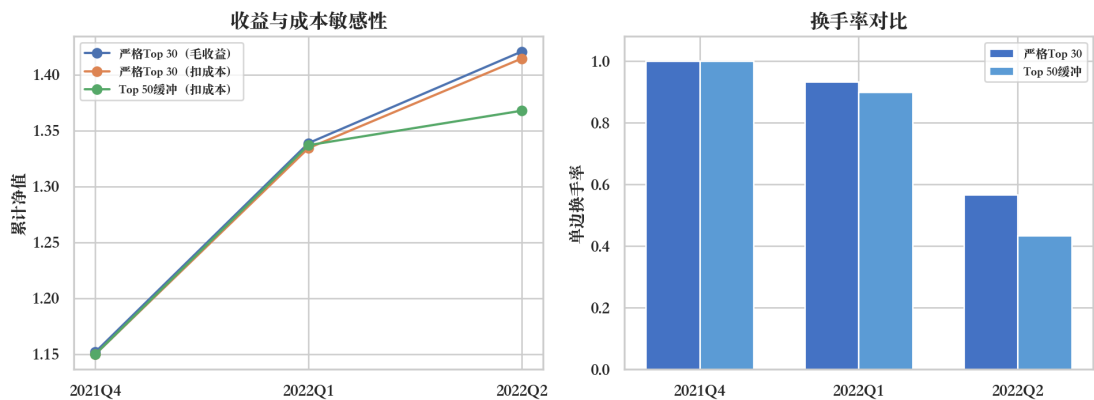


图 7 比较严格 Top 30 的毛收益、扣成本收益和 Top 50 缓冲换仓结果。严格 Top 30 的平均单边换手率为 83.33%，扣除单边 20bp 后，三季累计收益由 42.08% 降至 41.46%。Top 50 缓冲规则保留上期持仓中仍位于预测前 50 的股票，并用新股补足 30 只，平均换手率下降到 77.78%，但扣成本累计收益也下降到 36.79%。该测试期内的成本降低幅度小于排名纯度下降带来的收益损失，因此主策略继续使用严格 Top 30。

图8：正则化线性模型与树模型的特征依赖

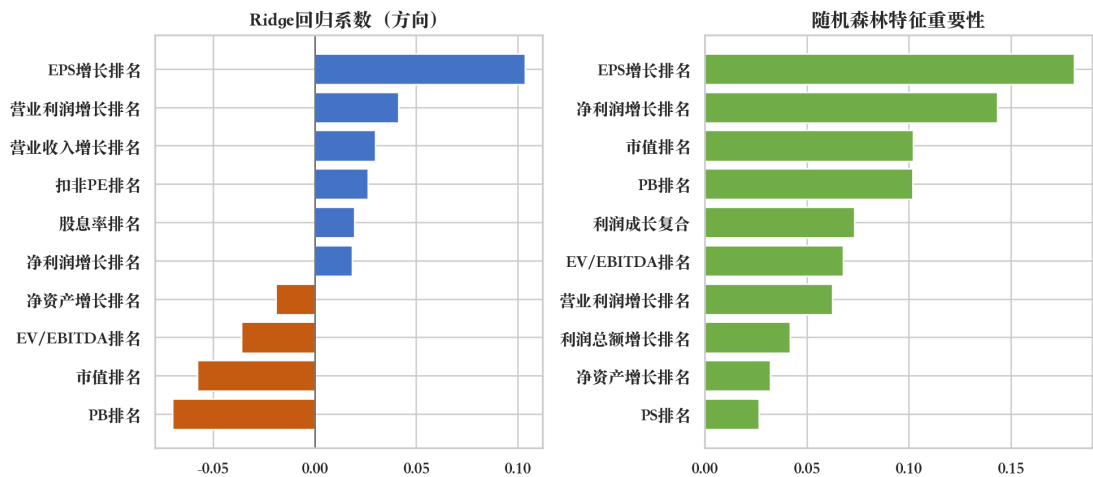


图 8 左侧使用 Ridge 系数表示特征方向，正值表示因子排名升高与预测收益排名升高同向，负值表示反向。正则化能够抑制高相关因子下过度放大的系数，右侧随机森林重要性则表示特征在树分裂中的使用程度。两种表达都显示模型较多使用盈利和成长信息，与主样本的财务因子设计一致；系数和特征重要性只描述模型依赖，无法据此识别因果效应。

6. 线性模型已能表达主要排序信息

线性回归和 Ridge 的验证与测试排序指标几乎相同，表明当前 23 项排名特征中的主要信息已经可以由较简单的线性组合表达。逻辑回归的测试 AUC 为 0.633、平均 Rank IC 为 0.263，说明“收益是否高于季度中位数”的分类概率也能用于股票排序。决策树的测试 IC 和组合收益在六种模型中较低，反映单棵树对样本分割较敏感；随机森林和梯度提升缓解了单树波动，但训练期验证 IC 并未显示出稳定的复杂模型优势。

主策略选择线性回归，依据是训练期内部验证 Rank IC 与 0.01 简约容差，并未使用测试期组合收益挑选模型。随机森林在三个测试季度中的 51.39% 毛收益只是样本外对照结果，不用于事后替换主模型。这一取舍保留了测试集的独立评价作用，也使后续 EW 和 PW 权重扩展都建立在同一个预先确定的线性排序模型上。

四、附加题：平安银行机器学习择时

1. 数据、特征和最终模型

附加题用单股日线数据检验机器学习概率能否转换为动态仓位。数据文件名中使用了“平安集团”，但证券代码 000001.SZ 对应平安银行，因此案例按证券代码记为平安银行。原始日线覆盖 2024 年 1 月 2 日至 2025 年 7 月 17 日，共 372 个交易日，能够覆盖上涨、回落和横盘阶段，但市场状态和样本长度仍然有限。

最终 X 只保留 4 项特征：5 日收益率表示近期动量，MA20 偏离率表示收盘价相对 20 日移动平均线的高低，RSI14 是用 14 日涨跌幅计算的相对强弱指标并缩放到 0 至 1，20 日成交量比是当日成交量与过去 20 日平均成交量的比值。 Y 定义为未来 3 个交易日收益是否大于 0，取值 1 表示上涨，取

值 0 表示未上涨。特征数量的缩减来自前期失败结果，目的是减少短样本下的重复信息和非线性过拟合。

可用建模样本按 70% 与 30% 严格按时间划分，并在分界处清除 3 行，使最后一个训练标签的结束日早于测试起点。最终模型为 180 行滚动逻辑回归，“滚动”表示每到一个新交易日，只使用当时已经可得标签的最近 180 条数据重新拟合，再预测当日概率。这种方式使特征、标签和预测严格按时间对齐，同时允许模型逐步吸收近期数据。

```
[8]:
addon_quality = json.loads(
    (ADDON_DIR / "metadata" / "data_quality_report.json").read_text(encoding="utf-8")
)
addon_metadata = json.loads(
    (ADDON_DIR / "metadata" / "model_run.json").read_text(encoding="utf-8")
)
addon_metrics = pd.read_csv(ADDON_DIR / "processed" / "additional_model_metrics.csv")

addon_quality_table = pd.DataFrame({
    "项目": ["原始交易日", "可用建模行", "训练行", "边界清除行", "测试行", "重复日期", "OHLC 逻辑异常"],
    "结果": [addon_quality["rows"], addon_quality["usable_model_rows"], addon_quality["train_rows"],
    ↪ addon_quality["purged_boundary_rows"], addon_quality["test_rows"], addon_quality["duplicate_dates"],
    ↪ addon_quality["ohlc_inconsistent_rows"]],
})
print("表 8: 附加题数据检查与时间划分")
display(addon_quality_table)

addon_model_table = addon_metrics[["model_label", "test_auc", "accuracy", "balanced_accuracy", "brier"]].copy()
addon_model_table.columns = ["模型", "静态模型测试 AUC", "准确率", "平衡准确率", "Brier 分数"]
print("表 9: 相同 3 日标签和 4 特征下的三种静态分类模型")
display(addon_model_table)
print(f"最终 180 行滚动逻辑回归: 验证 AUC={addon_metadata['final_validation_auc']:.3f}, 测试
↪ AUC={addon_metadata['final_test_auc']:.3f}。")
```

表 8: 附加题数据检查与时间划分

	项目	结果
0	原始交易日	372
1	可用建模行	350
2	训练行	241
3	边界清除行	3
4	测试行	106
5	重复日期	0
6	OHLC 逻辑异常	0

表 9: 相同 3 日标签和 4 特征下的三种静态分类模型

	模型	静态模型测试 AUC	准确率	平衡准确率	Brier 分数
0	逻辑回归	0.5202	0.5094	0.5180	0.2549
1	决策树	0.5088	0.4623	0.5205	0.2566
2	随机森林	0.5594	0.5094	0.5543	0.2493

最终 180 行滚动逻辑回归: 验证 AUC=0.631, 测试 AUC=0.571。

图9：平安银行价格、趋势与RSI过滤指标



图 9 显示日线样本经历了上涨、回落和横盘，MA5 与 MA20 在这些阶段多次交叉，RSI14 也在强弱区间变化。MA5 和 MA20 用于构造一个不依赖机器学习的均线对照策略，RSI14 则用于限制模型在过热区入场。在相同的 3 日 Y 和 4 项 X 下，表 9 中逻辑回归、决策树和随机森林的静态测试 AUC 分别为 0.520、0.509 和 0.559，都略高于 0.5；180 行滚动逻辑回归的测试 AUC 进一步达到 0.571。这一结果表明正类概率具有弱的正向排序能力，但 AUC 距离 1 仍较远，因此后续仓位规则不宜把概率视为确定预测。

2. 课程方法将概率转换为双阈值和动态仓位

模型概率表示对未来 3 日上涨可能性的估计，不会直接给出投入资金比例。仓位是持有股票的资金占组合总资产的比例，例如仓位 0.8 表示 80% 资金持有股票，其余 20% 保留为现金。课程方法使用一个买入阈值和一个卖出阈值，概率高于买入阈值时允许建仓，低于卖出阈值时清仓，两个阈值之间保持原仓位。这种双阈值设计可以减少概率在单一分界附近波动造成的反复交易。

买入阈值在 0.55、0.60 和 0.65 中选择，卖出阈值在 0.35、0.40 和 0.45 中选择，最大仓位在 0.6、0.8 和 1.0 中选择，共形成 27 种参数组合。参数只在训练期内部验证段上按夏普比率比较，入选值为买入 0.60、卖出 0.35 和最大仓位 0.8。当概率超过 0.5 时，目标仓位按概率超出部分线性增加，并受最大仓位约束：

$$Position_t = \min(MaxPos, \max(0, (p_t - 0.5) \times 2 \times MaxPos))$$

入场还要求 RSI14 低于 0.70，以减少价格过热阶段的追高交易。MA5 高于 MA20 的趋势过滤没有进入最终规则，因为模型特征已经包含趋势信息，再次过滤会减少有效交易机会。风险控制设置 8% 止损和 15% 止盈，每次仓位变动均按单边 20bp 扣费，使策略收益同时反映信号、暴露和交易成本。

[9]:

```

signal = addon_metadata["signal"]
signal_table = pd.DataFrame({
    "买入阈值": [signal["buy_threshold"]],
    "卖出阈值": [signal["sell_threshold"]],
    "最大仓位": [signal["max_position"]],
    "RSI 入场限制": ["RSI14 < 0.70"],
    "止损": [signal["stop_loss"]],

```

```

    "止盈": [signal["take_profit"]],
    "单边成本": [signal["transaction_cost"]],
})
print("表 10: 附加题最终交易规则")
display(signal_table)

```

表 10: 附加题最终交易规则

买入阈值	卖出阈值	最大仓位	RSI 入场限制	止损	止盈	单边成本
0.6000	0.3500	0.8000	RSI14 < 0.70	0.0800	0.1500	0.0020

图10: 逻辑回归概率、双阈值与实际仓位

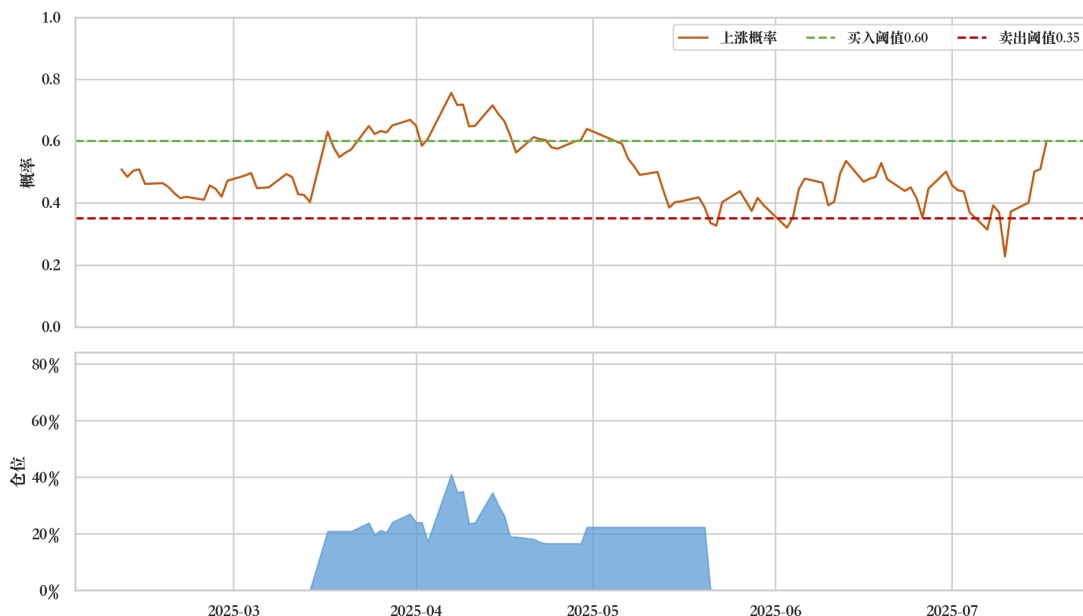


图 10 将滚动逻辑回归概率、买卖阈值和最终仓位放在同一时间轴上，可以直接检查概率是如何转换为资金暴露的。测试期共出现 1 次从空仓进入持仓，之后因模型概率低于 0.35 而退出，期间没有触发 8% 止损或 15% 止盈。持仓日占比为 39.45%，说明大部分测试日的概率未能持续越过买入阈值，双阈值因而使策略保留了较多现金。这一低暴露特征会同时降低收益弹性和回撤，所以后续多策略比较必须同时报告收益、波动和持仓比例。

3. 多策略回测比较

附加题设置买入持有和均线策略作为两个简单对照，以判断机器学习信号是否带来额外价值。买入持有在测试期开始建仓并持有到结束，代表不进行择时的基准；均线策略在 MA5 高于 MA20 时持有 80% 仓位，其余时间空仓，代表只使用简单趋势信息的规则。ML 择时策略使用上述滚动逻辑回归概率和动态仓位。三种策略都在仓位变化时扣除单边 20bp 成本，因此收益差异同时包含信号质量、平均暴露和换手频率的影响。

```

[10]: addon_strategy_metrics = pd.read_csv(
    ADDON_DIR / "processed" / "additional_strategy_metrics.csv"
)
addon_strategy_table = addon_strategy_metrics[[
    "strategy_label", "total_return", "annualized_return", "sharpe", "max_drawdown",
    "trade_count", "total_turnover", "days_in_market_ratio"
]].copy()
addon_strategy_table.columns = ["策略", "累计收益", "年化收益", "夏普比率", "最大回撤", "入场次数", "总换手", "持仓日占比"]
print("表 11: 附加题三种策略的测试期结果")
display(addon_strategy_table)

```

表 11: 附加题三种策略的测试期结果

	策略	累计收益	年化收益	夏普比率	最大回撤	入场次数	总换手	持仓日占比
0	ML 择时策略	0.0050	0.0117	0.5668	-0.0137	1	1.4512	0.3945
1	买入持有	0.0993	0.2446	1.3134	-0.1061	1	1.0000	0.9908
2	均线策略	0.0968	0.2382	1.8647	-0.0378	3	4.0000	0.7064

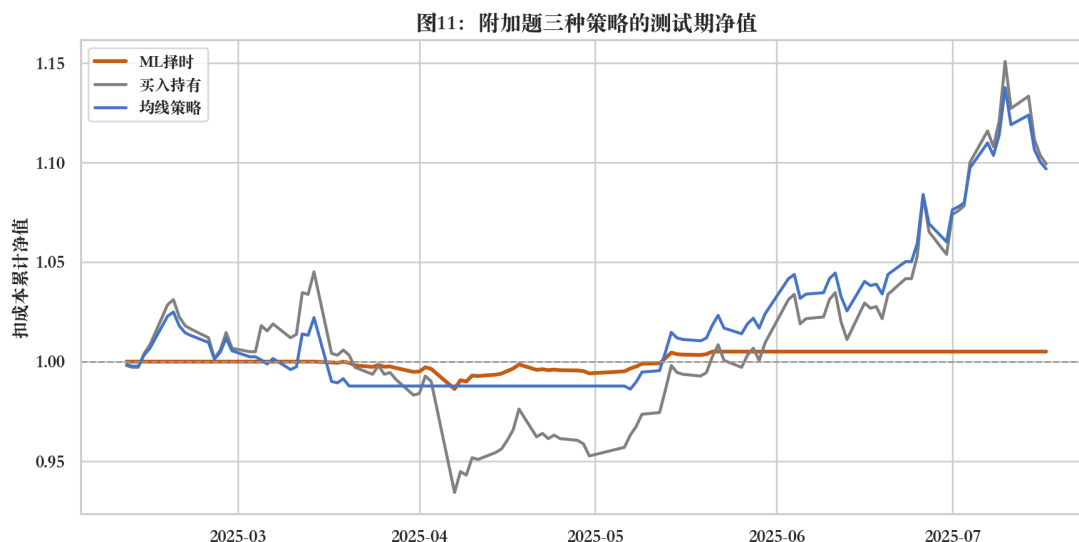


图 11 显示，在 109 个回测交易日中，ML 择时策略扣成本后收益为 0.50%，买入持有和均线策略分别为 9.93% 和 9.68%。ML 策略的年化波动率仅 2.09%，低收益与低波动同时出现，这与其 39.45% 的持仓日占比一致。最终 ML 策略虽然获得正收益，但未超过两个简单对照，因此当前证据只支持“概率具有弱排序信息”，不支持“机器学习策略收益更高”的判断。

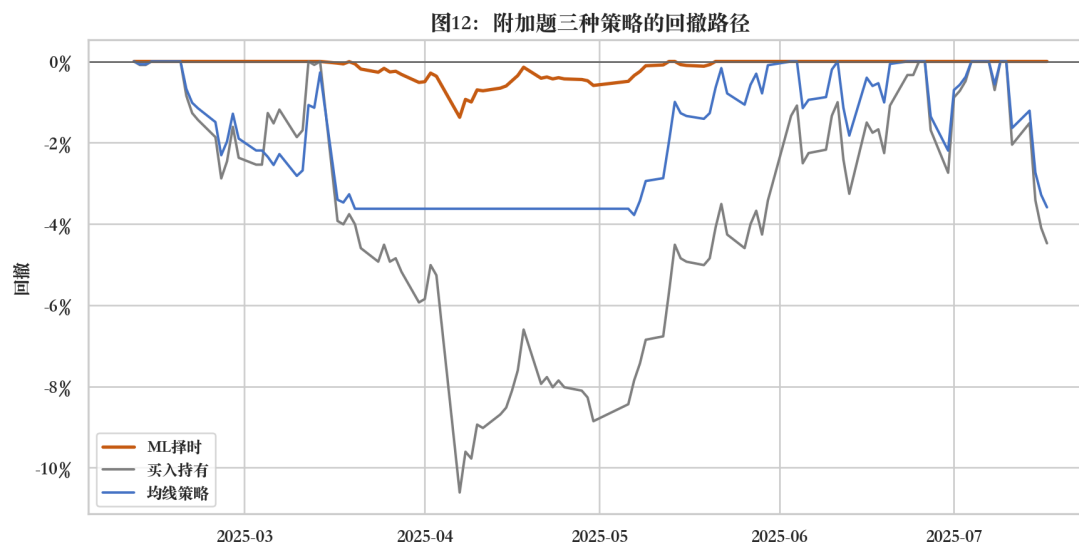


图 12 补充了净值比较中的下跌风险。买入持有的最大回撤为-10.61%，均线策略为-3.78%，ML 策略为-1.37%，风险排序与三种策略的市场暴露程度相符。ML 策略的较小回撤主要来自大部分时间持有现金，尚不能证明模型成功识别了所有下跌阶段。若要区分低暴露与真实择时能力，还需在更长时间和多种市场状态下比较相同平均仓位的基准。

五、预测加权提高收益，也增加波动和换手

1. EW 与 PW 组合

等权组合简称 EW，每只入选股票的权重相同；预测加权组合简称 PW，本作业根据模型预测排名分配权重，排名越靠前，权重越高。PW 可以放大高排名股票对组合的影响，也可能导致资金集中，因此同时设置单股权重上限。加权组合的换手率定义为相邻两期所有股票权重变化绝对值之和的一半，首期建仓记为 100%，净收益仍按毛收益减去 0.002 乘以换手率计算。

权重方案只使用 2021Q1 至 2021Q3 的三个训练期内部验证季度选择，网格包括持股数、加权方式、权重幂次和单股上限。参数确定后才一次性应用到 2021Q4 至 2022Q2 的三个测试季度，使 EW 与 PW 的差异来自预先确定的权重规则。由于内部验证和最终测试都只有三季，网格只能提供教学性敏感度比较，不代表权重参数已经稳定。

```
[11]: ENHANCED_DIR = PROJECT_ROOT / "data" / "task6" / "enhanced"
weighted_metrics = pd.read_csv(ENHANCED_DIR / "processed" / "main_weighted_strategy_metrics.csv")
weighted_returns = pd.read_csv(ENHANCED_DIR / "processed" / "main_weighted_quarterly_returns.csv", parse_dates=["Date"])
weight_grid = pd.read_csv(ENHANCED_DIR / "processed" / "main_weight_grid.csv")
auc_grid = pd.read_csv(ENHANCED_DIR / "processed" / "additional_guarded_auc_grid.csv")

weight_summary = weighted_metrics[["portfolio_label", "top_n", "weight_method", "weight_cap", "total_return",
    ↪ "annualized_volatility", "sharpe", "average_turnover"]].copy()
print(f"已加载{len(weight_grid)}组权重网格和{len(weighted_metrics)}个测试组合。")
```

已加载 84 组权重网格和 3 个测试组合。

表 13: EW/PW 与验证集选定组合的测试结果

组合	持股数	权重方式	累计净收益	年化波动率	平均换手率
EW Top30	30	等权	41.46%	11.03%	83.33%
PW Top30	30	排名加权	46.28%	15.68%	85.95%
验证集选定 PW20	20	排名平方加权	46.34%	18.75%	87.95%

图14: 等权EW与预测加权PW组合对比

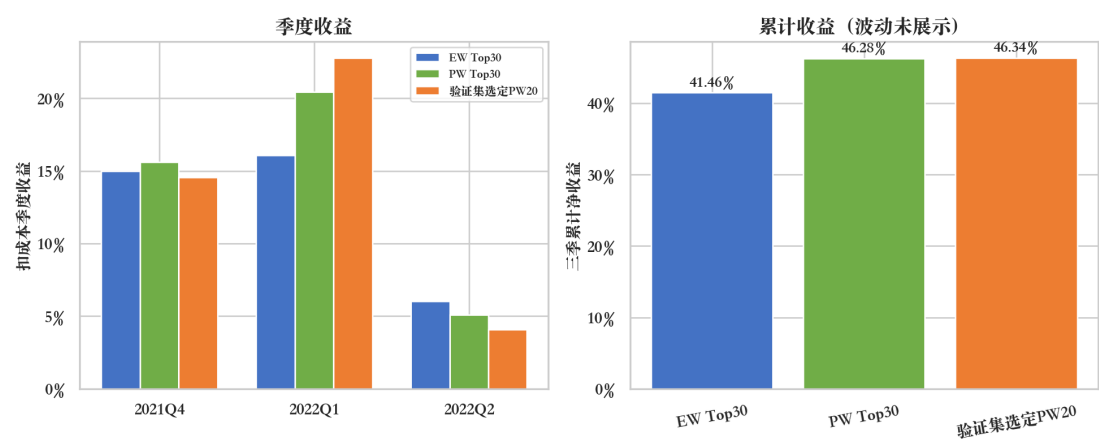


图 14 显示，EW Top30 扣成本后的三季累计收益为 41.46%，PW Top30 为 46.28%，收益差为 4.82 个百分点。验证集选出的方案为排名平方加权 PW Top20，单股上限 8%，其测试累计收益为 46.34%，只比 PW Top30 高 0.06 个百分点。PW Top30 的年化波动率由 11.03% 升至 15.68%，平均换手率也由 83.33% 升至 85.95%，说明收益增量伴随更高的风险和交易强度。测试期只有三季，因此该结果只能作为权重敏感性证据；结构简单、集中度较低的 EW Top30 仍作为正式主策略，PW 作为增强对照。

2. 模型保存和展示工具

.pkl 是用 Python pickle 格式序列化的对象文件，可以保留已拟合估计器的参数和结构。本作业保存的模型包同时记录估计器、特征名称、目标定义、训练时间和交易成本参数，使后续加载时可以同时核对模型口径。滚动逻辑回归在每个预测日都会重新拟合，因此不存在唯一的静态最终估计器；其.pkl 保存估计器模板、4 项特征和 180 行窗口配置，实际预测时仍只用当时已经可得的标签重新拟合。

Task6_Dashboard.html 将主任务、附加题、调参结果和最终选股汇总为离线只读看板。CSV_Regression_Tool.html 是纯前端回归工具，数据只在浏览器内存中处理，可选择 X 、 Y 和线性或逻辑回归，并下载预测 CSV 与模型 JSON。浏览器无法原生生成与 scikit-learn 兼容的 Python pickle，所以前端工具使用 JSON 保存系数和预处理参数，正式.pkl 继续由 Python 建模脚本生成。

六、主策略选择线性回归 EW Top30

主任务根据训练期内部 Rank IC 和 0.01 简约容差选择线性回归，交易规则为每季等权持有预测前 30 只。最近测试季度 2022Q2 的持仓为：300105、000806、002696、600287、600387、002358、603586、002478、002193、002699、603003、300533、600780、603086、600565、000726、000543、605088、002661、003042、002871、000616、002856、300495、000159、000892、603585、002869、002718、603329。这些代码依次对应预测排名第 1 至第 30，表 5 给出了其他两个测试季度的完整清单。

线性回归 EW Top30 扣除单边 20bp 成本后，三个测试季度的累计收益为 41.46%，市场等权同期为 -15.91%。PW Top30 的累计净收益提高到 46.28%，同时年化波动率由 11.03% 升至 15.68%，平均换手率也由 83.33% 升至 85.95%。测试窗口只有三季，收益增量尚不足以证明 PW 在新时期稳定占优，因此 EW Top30 保留为规则更简单、集中度较低的主策略，PW 仅用于展示权重调整的敏感性。

附加题的 180 行滚动逻辑回归测试 AUC 为 0.571，说明正类概率的排序方向正确，但信号强度较弱。转换为双阈值和动态仓位后，ML 择时策略的净收益为 0.50%，低于买入持有的 9.93% 和均线策略的 9.68%。当前数据因而支持主任务线性回归 Top30 的选股结论；附加题只能说明模型含有弱预测信息，尚不能替代简单基准。

七、思考：验证高分未能稳定转化为测试优势

最初方案使用 5 日标签、9 项技术特征和静态随机森林，验证 AUC 为 0.756，测试 AUC 却只有 0.335。AUC 衡量模型能否把未来上涨样本排在下跌样本前面，0.5 相当于随机排序，越接 1 说明排序越准。测试 AUC 低于 0.5 表示模型到新时期后不仅失去了预测能力，还出现了明显的排序反向。验证高分与测试低分相差 0.421，这个反差成为后续三轮调整的起点。

出现 AUC 低于 0.5 时，不能立即归因于市场难以预测，因为标签写反、概率列取错或时间对齐失误也会得到类似结果。检查确认标签 1 确实表示未来上涨，评价使用的也是类别 1 的概率，每个交易日的特征只与该日之后的收益配对。另外使用一个按正负样本排名的独立公式重新计算 AUC，结果与 scikit-learn 完全一致。这些检查说明 0.335 不是简单的编程方向错误。

为了确认整个建模流程确实能学到规律，又安排了两个对照检验。将上涨与下跌标签随机打乱后，平均 AUC 回到 0.5 附近，这与没有可学习关系时的预期相符。人工构造一个能由特征明确推断的标签后，AUC 又高于 0.9，说明数据处理和模型评价流程具备正常的学习能力。排除基本实现问题后，更合理的解释是：第一轮在训练和验证阶段学到的复杂关系，到测试阶段已经发生变化。

理论上，如果模型在多个独立时期都稳定得到低于 0.5 的 AUC，可以尝试将概率转换为 $1-p$ ，把稳定的反向关系转换为正向信号。本次的 0.335 只出现在一个已被查看的测试窗口，还没有证据说明反向关系会在下一阶段持续。看完测试结果再把概率取反，实质上是利用测试集调整策略，会让新的分数失去独立检验的意义。因此没有采用简单取反，而是从预测目标、特征数量和模型结构三个方面重新设计。

第二轮将预测目标 Y 由未来 5 日收益缩短为未来 3 日收益，并将输入特征 X 精简为 5 日收益、MA20 偏离率、RSI14 和 20 日成交量比。更短的目标与短期择时操作更接近，也能减少相邻样本之间重复使用同一段未来收益的问题。特征数量从 9 项减少到 4 项，相当于降低模型在短样本中记住偶然波动的机会。第二轮静态逻辑回归的测试 AUC 回到 0.520，虽然只比随机水平高 0.020，但至少说明排序方向已经恢复为正。

第三轮使用 180 行滚动逻辑回归，即每到一个新交易日，都使用最近 180 条当时已经可得的数据重新估计模型。这种方式让新的市场信息更快进入模型，测试 AUC 也由 0.520 提高到 0.571。与此同时，滚动窗口会舍弃较早数据，样本数量减少可能让估计结果更容易波动。因此 0.571 只能解释为当前测试窗口内出现弱正向排序，还不能证明模型已经稳定适应市场变化。

三轮调整后又完成 144 组参数组合的网格搜索。验证集上分数最高的组合达到 0.752，但它的测试 AUC 只有 0.538，低于原滚动模型的 0.571。这个结果说明，当验证时间较短、候选组合又很多时，找到的“最高分”可能只是偶然适合这一小段历史数据，这种现象称为验证集过拟合。最终保留结构较简单的原滚动逻辑回归，避免根据已经看到的测试分数反复更换模型。

调参过程还提醒了预测指标与投资结果之间的差别。滚动逻辑回归的 AUC 为 0.571，说明上涨概率含有一定的排序信息；转换为双阈值和最大 80% 动态仓位后，持仓日占比只有 39.45%，净收益也只有 0.50%，低于买入持有的 9.93%。AUC 只回答“排序是否较准”，最终收益还取决于何时买入、持有多少仓位、保持多长时间和支付多少交易成本。后续实验要在查看结果前固定标签、特征、滚动窗口和买卖阈值，再用一段全新数据同时检验 AUC、扣成本收益和相同平均仓位下的基准，才能判断这个弱信号能否重复。

[12]:

```
tuning_rounds = pd.read_csv(ADDON_DIR / "processed" / "additional_tuning_rounds.csv")
tuning_table = tuning_rounds[["round", "design", "validation_auc", "test_auc", "result"]].copy()
tuning_table.columns = ["轮次", "设计", "验证 AUC", "测试 AUC", "结果"]
print(f"已加载{len(tuning_table)}轮调参记录和{len(auc_grid)}个最终 AUC 对照方案。")
```

已加载 3 轮调参记录和 2 个最终 AUC 对照方案。

表 14：三轮调参记录

轮次	设计	验证 AUC	测试 AUC
1	5 日标签、9 特征、静态随机森林	0.756	0.335
2	3 日标签、4 特征、静态逻辑回归	0.692	0.520
3	3 日标签、4 特征、180 行滚动逻辑回归	0.631	0.571

表 15：144 组受控网格搜索的最终对照

方案	特征	C	窗口	验证 AUC	测试 AUC
网格验证冠军	趋势 4 因子	0.001	60	0.752	0.538
原滚动逻辑回归	精简 4 因子	0.1	180	0.623	0.571

图13: 三轮调参与分类模型对比

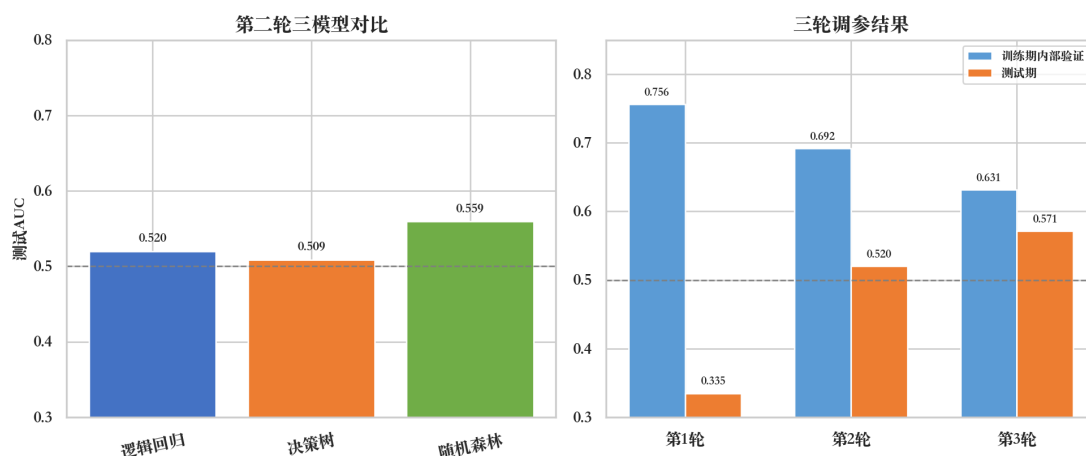


图 13 左侧显示，第二轮三种静态模型的测试 AUC 介于 0.509 和 0.559 之间，模型复杂度没有形成单调改善。右侧三轮记录显示，第一轮验证 AUC 最高，测试 AUC 却最低；随着标签缩短、特征精简和滚动更新，验证 AUC 从 0.756 降至 0.631，测试 AUC 则从 0.335 升至 0.571。验证分数与测试表现的反向变化说明，单个短验证窗口不足以识别稳定模型，特征和结构取舍需保留对新时期数据的独立检验。

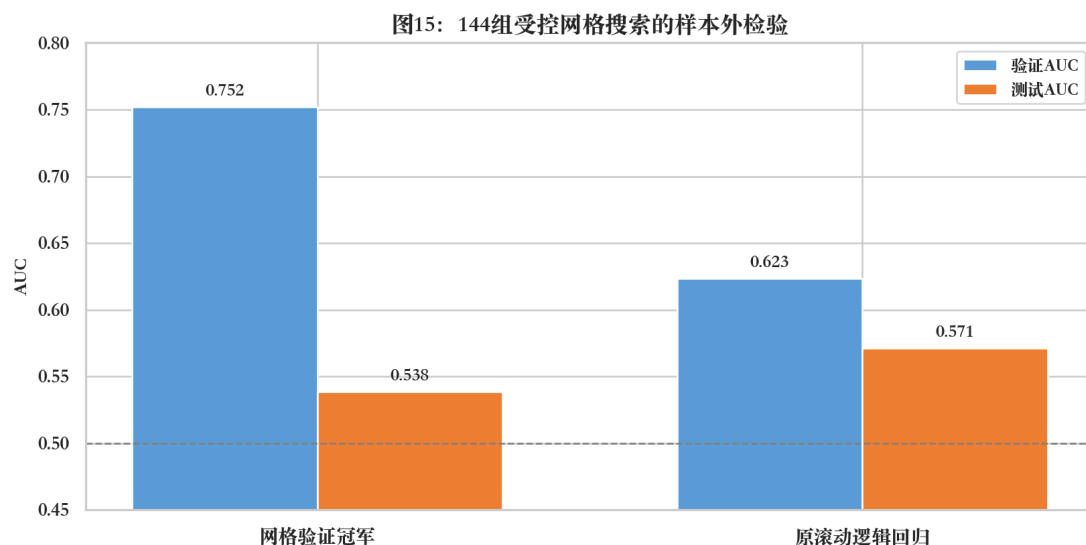


图 15 进一步显示，网格验证冠军将验证 AUC 提高到 0.752，测试 AUC 却只有 0.538；原滚动逻辑回归的验证 AUC 为 0.623，测试 AUC 为 0.571。两个方案的差异说明，扩大搜索范围可以提高对已定义验证窗口的匹配，却不能保证市场关系在下一阶段继续成立。本作业已多次查看同一测试窗口，所以调参后的 AUC 均应解释为探索性结果。后续检验应锁定特征、窗口和阈值，再使用完全未参与调整的新时期数据评价。